

# Mitchell C. Elliott

(408) 643-4372 | [mitch.elliott@pacbell.net](mailto:mitch.elliott@pacbell.net)

[linkedin.com/in/mitchellelliott18](https://www.linkedin.com/in/mitchellelliott18) | [github.com/melliott18](https://github.com/melliott18) | [melliott18.github.io](https://melliott18.github.io)

Systems Researcher | San Francisco Bay Area | U.S. Citizen

## SUMMARY

---

Systems researcher studying the carbon, energy, cost, and performance tradeoffs of data-center storage. Research combines lifecycle analysis, trace-driven simulation, workload characterization, and reproducible experimentation to evaluate heterogeneous storage architectures. Earned an M.S. in Computer Science at the University of California, Santa Cruz, and led a 3-person team building shared research infrastructure. Teaching experience covers computer system design, concurrency, networking, memory management, testing, and debugging; industry experience spans cloud operations, internal applications, and operational analytics.

## EDUCATION

---

### University of California, Santa Cruz

Sept. 2022 – Dec. 2024

*M.S. Computer Science*

Santa Cruz, CA

- **Coursework:** Advanced Operating Systems, Artificial Intelligence, Algorithms, Archival Storage System Design, Computer Architecture, Computer Security, Cryptography, Database Systems, Distributed Systems, Independent Study, Programming Languages

### University of California, Santa Cruz

Sept. 2018 – June 2022

*B.S. Computer Science*

Santa Cruz, CA

- **Coursework:** Artificial Intelligence, Algorithms, Assembly Language, C Programming, Compiler Design, Computer Architecture, Computational Models, Computer Networks, Computer Security, Computer System Design, Data Structures, Database Design, Distributed Systems, Embedded Operating Systems, Functional Programming, Network Programming, SQL, Web Applications
- **Honors:** Dean's List, Fall 2019, Winter 2020, and Spring 2020

## WORK EXPERIENCE

---

### UCSC Center for Research in Systems and Storage

Jan. 2026 – Aug. 2026

*Research Software Engineer, Platform Infrastructure*

Santa Cruz, CA

- Led a 3-person engineering team developing and operating shared infrastructure for systems research, coordinating platform architecture, implementation, code review, and experiment support across parallel lab projects
- Built and maintained containerized research environments on NRP Nautilus Kubernetes with Ceph-backed persistent storage for repeatable compute- and data-intensive workloads across multiple users
- Standardized deployment, configuration, data-access, and experiment workflows across Kubernetes, Ceph, BigQuery, DuckDB, and Parquet to reduce environment-specific setup and improve research reproducibility
- Developed reusable self-service operational tooling for launching, monitoring, and managing long-running experiments, improving resource utilization while avoiding thousands of dollars in equivalent GCP compute costs

### UCSC Center for Research in Systems and Storage

Dec. 2024 – Jan. 2026

*Research Software Engineer, Storage Systems*

Santa Cruz, CA

- Evaluated storage architectures across device classes and workload patterns, analyzing performance, capacity, energy, and cost tradeoffs to guide infrastructure design decisions
- Designed repeatable trace-driven workflows for comparing storage technologies under realistic demand, standardizing inputs, parameter sweeps, validation, and result review across configurations
- Collaborated with systems researchers to translate open-ended research questions into measurable experiments and synthesize results into storage-architecture recommendations

**University of California, Santa Cruz***Graduate Student Researcher*

Jan. 2024 – Dec. 2024

Santa Cruz, CA

- Created CarbonStream, an end-to-end total cost of carbon analyzer for data centers, for M.S. research
- Evaluated the environmental impact of data-center storage technologies through lifecycle analysis
- Developed a storage-optimization framework integrating cost, performance, and sustainability metrics
- Identified opportunities to improve hybrid storage energy efficiency through workload-specific strategies

**Perfectly Snug**

July 2023 – Sept. 2023

*Software Engineer Intern*

Remote

- Developed a material requirements planning (MRP) application using Python, Node.js, React, and SQL, integrating inventory, customer orders, bills of materials, and production data into a single internal system
- Built dashboards for inventory availability, material requirements, order status, and production progress; automated reconciliation and scheduling, reducing manual planning time by roughly 50%
- Worked directly with manufacturing stakeholders to gather requirements, iterate on operational workflows, debug edge cases, and translate production needs into maintainable internal software
- Developed dashboards and reports using Shopify API data to support daily manufacturing build plans
- Integrated product-testing software into a mobile application to streamline the build procedure

**ParkourSC, Inc.**

July 2022 – Sept. 2022

*DevOps Engineer Intern*

Remote

- Operated production cloud and platform infrastructure for a supply-chain application deployed across 100+ Kubernetes clusters, spanning IoT ingestion, Kafka, Spark, Cassandra, Elasticsearch, and web services
- Created and tuned New Relic alerting policies using NRQL, reducing false-positive production alerts by approximately 80% across distributed services
- Worked with developers to improve incident resolution and alert-tuning workflows, standardizing how alerts were reviewed, adjusted, and escalated during production operations
- Automated monitoring for Kubernetes continuous delivery and cluster upgrades, integrating New Relic and Jira to suppress maintenance-related alerts while preserving operational context
- Investigated outages across application, platform, and cloud infrastructure layers by tracing deployment topology and service dependencies and correlating Kubernetes state, dashboards, metrics, logs, alert incidents, and NRQL queries across Spark, Kafka, Cassandra, and Elasticsearch

**uLab Systems, Inc.**

June 2021 – Sept. 2021

*Site Reliability Engineer Intern*

San Mateo, CA

- Migrated a production application from Lightsail to EC2 and operated its AWS infrastructure with CloudWatch instance-health monitoring, improving deployment flexibility, operational control, and maintainability
- Built a secure AWS environment using VPC, Elastic Load Balancing, and IAM access controls, managing network isolation and permissions for cloud resources and services
- Integrated Datawiza Access Broker with Okta, implementing centralized authentication and proxy-based access controls for the production application while filtering unwanted web traffic
- Wrote a Python script to copy objects and preserve Access Control Lists (ACLs) between Amazon S3 buckets, automating data migration while retaining object-level permissions
- Designed and implemented a three-stage CI/CD pipeline using GitHub, AWS CodeBuild, and Amazon ECS for development, validation, and production, reducing manual release work and improving deployment consistency
- Used Terraform for repeatable AWS infrastructure provisioning and configuration, and documented provisioning, deployment, and recovery workflows in operational runbooks

**Nevtec, Inc.**

June 2019 – Sept. 2019, June 2020 – Sept. 2020

*Data Analyst Intern*

San Jose, CA

- Created a real-time ticket-monitoring system using Power BI and SQL against ConnectWise service data, building operational dashboards for Nevtec's CEO and IT support team
- Wrote Power BI Data Analysis Expressions (DAX) scripts to create calculated columns and transform operational service datasets
- Analyzed service-ticket data to compare engineer efficiency, customer IT request patterns, and time spent resolving different types of support tickets
- Built dashboards reporting ticket response and closing times, billable hours, and project time utilization
- Provided dashboard metrics showing when service-level-objective policies were met or violated

**Nevtec, Inc.**

June 2018 – Sept. 2018

*Computer Technician Intern*

San Jose, CA

- Configured dozens of customer PCs and automated their setup using Windows Server, PowerShell, XML, and batch scripting, eliminating manual installation clicks and configuration of custom options
- Reduced setup time for an entire batch of PCs to the order of minutes, compared with several hours when machines were configured individually
- Documented tables and their relationships in Nevtec's ConnectWise database, extending the limited basic vendor documentation and public information previously available
- Cataloged previously undocumented custom data in the ConnectWise database to support internal understanding of enterprise datasets

## TEACHING EXPERIENCE

---

**University of California, Santa Cruz**

Jan. 2023 – Dec. 2024

*Head Teaching Assistant – CSE 130: Principles of Computer System Design*

Santa Cruz, CA

- Served as Head Teaching Assistant for CSE 130 across six quarters, mentoring teaching staff and supporting an upper-division computer system design course serving 1,000+ students
- Coordinated course delivery and staff responsibilities around competing priorities and scheduling conflicts; standardized documentation, grading workflows, issue escalation, onboarding, and staff coverage
- Built and maintained GitLab CI/CD infrastructure for systems-programming assignments, automating compilation, testing, and feedback workflows and reducing estimated overall grading time by 50%
- Developed programming assignments and led discussions on concurrency, networking, memory management, and reliable C/C++ systems software; taught students to design clean, modular, well-organized programs and to use and create reusable libraries
- Taught students to build debugging and testing harnesses, identify edge cases, and write unit, regression, and end-to-end tests that verify specific program behavior
- Provided a subset of assignment tests and directed students to develop additional tests; individual cases targeted specific functionality or edge conditions, with most difficult cases hidden from students
- Guided runtime investigation with GDB, Linux perf, /proc inspection, shell piping, logging, and print debugging to understand execution behavior and identify performance bottlenecks
- Taught diagnosis of multithreading, synchronization, thread safety, command and string parsing, NULL pointer dereferences, network failures, improper file I/O, bad file descriptors, and errors reported by function and system-call return values

**University of California, Santa Cruz**

Apr. 2022 – June 2022

*Undergraduate Course Tutor*

Santa Cruz, CA

- Tutored students on programming assignments for CSE 130: Principles of Computer System Design
- Reviewed course material and taught practical programming and debugging techniques to students
- Assisted teaching assistants during discussion sections and office hours for systems coursework

# PROJECTS

---

## **SimOps Workbench: Simulation Experiment Dashboard**

June 2026

- Built a TypeScript and React self-service dashboard around a dynamics simulator, turning one-off runs into repeatable workflows for configuring scenarios, monitoring execution, and comparing telemetry
- Integrated Flask APIs and SQLite-backed experiment history with parameter-sweep and batch workflows, allowing completed experiments to be reopened, reproduced, and compared
- Added interactive telemetry comparisons, JSON and CSV exports, printable reports, and Docker Compose workflows to improve analysis, sharing, and reproducibility

## **Workload Trace Analysis Platform (UCSC CRSS)**

Jan. 2026 – Aug. 2026

- Built a data-processing platform for multi-month Google cluster and power traces stored in Google Cloud Storage, using BigQuery to select and extract bounded source datasets into repeatable local analysis workflows
- Materialized extracted traces as Parquet and used DuckDB to query, transform, join, and validate reusable datasets without repeatedly scanning the complete remote source
- Created CLI and automated testing workflows to orchestrate dataset selection, BigQuery extraction, Parquet conversion, DuckDB processing, schema validation, and packaging of simulator-ready inputs

## **CogniStore: AI-Powered Data Lifecycle Manager**

Sept. 2025 – Present

- Developed an intelligent data-management platform unifying file, block, and object storage behind a common API for lifecycle management across heterogeneous storage services
- Built a modular storage abstraction layer integrating PostgreSQL with pgvector, OpenSearch, and Redis for metadata persistence, semantic indexing, and low-latency retrieval
- Implemented an AI-driven policy engine for automated data placement across storage tiers based on lifecycle, cost, performance, and sustainability goals
- Used GitHub Actions in the project's CI/CD process to run automated tests

## **CarbonStream: Data Center Carbon TCO Simulator (UCSC CRSS)**

Jan. 2024 – Present

- Built a Python trace-driven simulator for evaluating performance, capacity, energy use, and carbon impact across heterogeneous data-center storage architectures
- Modeled workload behavior using device latency, throughput, power, lifespan, capacity, and regional carbon-intensity inputs under changing demand
- Implemented bottleneck detection, service-level-objective checks, configuration search, parameter sweeps, and automated CSV and visualization outputs for reproducible analysis
- Used perf, fio, gem5, and SimpleSSD in SSD simulation work and to prototype an early version of the CarbonStream TCO simulator; used Flamegraphs to visualize perf data
- Recorded SSD I/O traces with blktrace and used other workload traces in blktrace format as source inputs to the early CarbonStream simulator
- Used Grafana and Prometheus to view historical Wikipedia data for CarbonStream

## **Automated Grading Infrastructure** (UCSC CSE 130)

Jan. 2023 – Dec. 2024

- Maintained GitLab runner checks for Make compilation, clang-format, and required files. Maintained reusable CI YAML with one build stage, using TESTING\_DOCKER\_IMAGE to select the Docker image and CI\_USER, CI\_TOKEN, and CI\_REPO to clone centralized CI scripts
- Added a Python CI entrypoint that selected an assignment from Git history, integrated centralized grader scripts and recursive submodules, and propagated validation failures through the job exit status. Added a conditional grading bypass returning success when the history scan encountered a notes path before an assignment path
- Operated full assignment grading in Docker containers on a local Ubuntu VM because school IT servers lacked capacity. Automated repository preparation and full-grader execution with Python and Bash batch workflows, reducing estimated overall grading time by 50%
- Built a Python utility to compare grading runs, identify missing or higher, lower, and equal results, and summarize scores with mean, median, and mode using the statistics module
- Built a Python merger comparing two grading runs and selecting each student's entire higher-total rubric file, retaining entries present in only one run and reporting reconciliation decisions and statistics
- Operated score-processing workflows using the shared PLY lex/yacc parser to read structured grading rubrics, scores, and comments; automated publication of rubric scores and feedback to Canvas LMS through Python canvasapi integration and batch upload wrappers
- Authored a configurable C reader/writer-lock test harness using pthread mutexes and condition variables for coordinated starts and exclusion checks, with per-thread counts and reader/writer priority ratios. Expanded stress and priority testing through N=100 with 50 reader and 50 writer threads
- Expanded file-I/O tests for large and binary payloads, missing content lengths, and failed writes, checking output contents, diagnostics, and exit status. Corrected randomized request-size bounds in an existing LD\_PRELOAD/dlsym partial-I/O shim intercepting libc read, write, and close calls
- Added HTTP parsing and data-integrity tests for request syntax, headers, content lengths, and error cases using curl and a supplied raw TCP client. Collaboratively extended HTTP server-error tests with LD\_PRELOAD/dlsym filesystem-error shims and exact HTTP 500 response checks
- Added partial-GET/full-PUT and partial-PUT/full-GET interleaving workloads to concurrent HTTP extra-credit grading, reusing the existing workload execution, audit-order validation, and response-state checks
- Maintained Python assignment handlers and TOML configurations for required files, test preparation, rubrics, timeouts, and cleanup. Used Clang and Make to build C harnesses; maintained shared checks for Makefile targets, executables, warning flags, and compilation commands, and operated Valgrind memory-error and leak checks on submitted C programs
- Coauthored standalone queue pop-validity and push-order tests and added pthread linkage to concurrent-queue test wrappers
- Documented repository preparation, full and targeted grading, score review and upload, and commands for comparing and reconciling results

## **NRWLock: Thread-safe Reader-Writer Lock**

Oct. 2023 – Nov. 2023

- Developed a thread-safe reader-writer lock library in C using POSIX mutexes and condition variables
- Implemented configurable N-Way blocking for concurrent reader and writer access
- Designed priority scheduling policies with fairness, throughput, and starvation avoidance as goals

## **In-Memory File System**

Apr. 2022

- Built a Unix-style in-memory filesystem with a tree-structured hierarchy in C++
- Implemented command support in a shell environment using GNU Core Utilities-style semantics
- Managed an inode tree with root, directory, and file mappings for filesystem operations

## **Distributed Key-Value Store**

Oct. 2021 – Dec. 2021

- Built a Go in-memory key-value store with causal consistency, sharding, and replication
- Implemented reverse-proxy request forwarding, replica-to-replica HTTP handlers, and concurrent broadcast paths for replication and failed-replica view updates
- Implemented vector-clock operations and shard-aware routing that directed writes to the owning shard and replicated them within that shard

## DDFS: Deduplicating Block Storage System

May 2021 – June 2021

- Built a deduplicating block storage system for FreeBSD in C using hashed addressing
- Mapped 160-bit keys to unique 4 KiB blocks stored on a mounted memory disk
- Designed persistent metadata structures, inode-management workflows, and block-allocation logic; used a superblock to store filesystem metadata and inodes to contain mappings to data blocks
- Implemented address translation and data-retrieval logic below high-level storage APIs

## Unix Shell

Apr. 2021

- Created a Unix shell supporting single and piped commands, I/O redirection, and built-in commands
- Implemented return-value validation and error handling for command execution and malformed input
- Used Flex for token parsing alongside modular process-management and redirection logic

## MRPC: Multithreaded RPC Server

Oct. 2020 – Dec. 2020

- Implemented a multithreaded RPC server supporting concurrent socket-based client requests
- Designed a custom client-server protocol with requests and responses transmitted in network byte order
- Developed request processing with recursive name resolution, error handling, and persistent key-value storage
- Implemented RPC arithmetic operations and server-side file creation, reading, writing, and file-size queries

## Jake's Hockey Pool

June 2020 – Oct. 2020

- Created a live data-feed interface mapping to the undocumented NHL REST API
- Contributed to data structures and schema design for application state and pool workflows
- Applied modular programming practices to streamline development, testing, and architecture review

## LZW File Compressor

Dec. 2019

- Developed an LZW compression and decompression tool in C for lossless text and binary file handling
- Implemented buffered I/O and variable-length codes for file compression and decompression
- Structured the implementation with modular abstract data types for tries and word tables

## TECHNICAL SKILLS

---

**Languages:** Python, C, C++, Go, Java, JavaScript, TypeScript, Bash, PowerShell, SQL

**Systems:** Linux, FreeBSD, Windows Server, distributed systems, storage systems, POSIX synchronization, multithreading, socket programming, RPC, gRPC

**Cloud & Delivery:** AWS, GCP, Kubernetes, Docker, Terraform, GitHub Actions, GitLab CI/CD, Jenkins, AWS CodeBuild, Amazon ECS, Git, infrastructure automation

**Data & Storage:** PostgreSQL, pgvector, MySQL, SQLite, Redis, OpenSearch, Cassandra, DuckDB, Parquet, BigQuery, Google Cloud Storage, Amazon S3, MinIO, Ceph, RocksDB, ZFS, Kafka

**Applications & Testing:** React, Flask, FastAPI, Node.js, REST APIs, data modeling, automated testing, regression testing, end-to-end testing, Clang, Make

**Reliability & Performance:** New Relic, NRQL, CloudWatch, Prometheus, Grafana, GDB, Valgrind, perf, fio, blktrace

**Simulation & Analytics:** gem5, SimpleSSD, Jupyter, Pandas, Matplotlib, Seaborn, Power BI, DAX

## PRESENTATIONS

---

*Modeling Power Usage in Data Centers.*

CRSS Spring IAB Meeting, Santa Cruz, CA.

June 2026

*CarbonStream: An End-to-End Total Cost of Carbon Analyzer for Data Centers.*

CRSS Fall IAB Meeting, Santa Clara, CA.

Dec. 2025

*CarbonStream: An End-to-End Total Cost of Carbon Analyzer for Data Centers.*

CRSS Spring IAB Meeting, Santa Cruz, CA.

May 2025

*Analyzing the Total Cost of Ownership of Carbon in Data Centers.*

CRSS Weekly Seminar, Santa Cruz, CA.

Oct. 2024

*The Five-minute Rule Thirty Years Later and its Impact on the Storage Hierarchy.*

CRSS Weekly Seminar, Santa Cruz, CA.

Oct. 2023